

Reproducing the Speed, Utility, and Fairness of TABFAIRGDT

COMP5331 project type: Implementation

Group number: [Group number]

Members (name; student ID; research/FYP supervisor and topic, if applicable): [Member 1]; [Member 2]; [Member 3]; [Member 4]; [Member 5]; [Member 6, if applicable]

Relationship to members' research/FYP: [For each member, state whether a research/FYP topic exists and, if so, explain how this course project differs.]

Course-project declaration: This work is undertaken for COMP5331 and will be reported as a course project.

Project description

Motivation and objective

Synthetic tabular data can support data sharing, augmentation, and controlled evaluation, but useful synthetic data should preserve more than marginal feature distributions. If a generator reproduces biased relationships between a sensitive attribute and a prediction target, models trained on its samples may inherit unequal treatment. Conversely, a generator that achieves a favorable fairness measure by erasing information may damage downstream utility. TABFAIRGDT, proposed by Panagiotou and colleagues, uses autoregressive decision trees to generate fair tabular data quickly. Our implementation project will evaluate its three linked claims—speed, predictive utility, and fairness—under one coherent protocol on a small selection of public datasets.

The key question is whether TABFAIRGDT provides a better utility–fairness–runtime balance than uncomplicated alternatives when the same downstream learner and held-out real test data are used throughout. We will aim to reproduce the direction of the paper's reported comparisons and to quantify how sensitive that comparison is to fairness-strength settings and dataset preprocessing. This project does not seek to invent a new generator. Its contribution as a course implementation is a transparent, independently checkable evaluation of the published method, including intermediate data quality checks that explain the final classifier results.

Paper, code, and datasets

The principal paper is Emmanouil Panagiotou and coauthors, “TABFAIRGDT: A Fast Fair Tabular Data Generator using Autoregressive Decision Trees,” ICDM 2025. The author repository includes a usage example, main experiment script, dependency list, and evaluation or baseline modules. We will pin a commit and first run the provided small example to verify that generation and evaluation operate in our environment. We will then trace which repository settings correspond to the fairness intervention described in the paper. If the published implementation contains multiple parameterizations, we will choose one documented setting for the main comparison and reserve a small parameter sweep for analysis.

We will choose one or two public tabular classification datasets with a well-defined sensitive attribute and outcome, such as UCI Adult and a second dataset from the paper or OpenML. For each dataset we will record the downloaded version, source, allowed use, target label, sensitive attribute, field types, missing-value treatment, category encoding, and train/test split. We will fix the real-data test set before fitting any generator. All imputation, scaling, and category vocabularies used to train the generator or downstream model will be learned from the real training partition only. This separation prevents information from the held-out test set entering synthetic training data and makes downstream comparisons meaningful.

Evaluation protocol

Our main comparison will include a downstream classifier trained on the real training set, the same classifier trained on samples from TABFAIRGDT, and the classifier trained on synthetic samples from a simpler generator available in the codebase or easy to configure. We will keep the synthetic sample count equal across generators and use the same

downstream classifier family and tuning budget. If an additional published baseline requires substantially more compute or incompatible dependencies, we will substitute a transparent lightweight baseline and identify that departure. We will repeat experiments under several seeds, using saved real splits so that the generator is the principal changing factor.

On the held-out real test set, we will measure predictive accuracy and, where probabilities are meaningful, AUROC or macro-F1. For fairness, we will report demographic parity difference and equal opportunity difference by the chosen sensitive attribute, with the group denominators shown. These metrics answer different questions: parity concerns the rate of positive predictions, while equal opportunity conditions on true positives. Neither alone establishes an unqualified notion of fairness. We will present both, together with utility, so that one cannot mask a severe trade-off. We will also measure generator fitting time, sampling time per fixed number of records, and downstream training time. To investigate whether fairness improvements arise from dataset distortion, we will compare selected numeric distributions, category frequencies, target prevalence, and the number of synthetic records in each sensitive group.

The principal analysis will vary the method's fairness-strength parameter over a small prespecified set. We will plot utility against each fairness metric and annotate runtime, forming a compact trade-off picture rather than reporting a single favorable configuration. We will compare variability across seeds and note whether a group has too few test observations for a stable equal-opportunity estimate. In that case, we will report the count and uncertainty rather than treating a small numerical difference as reliable. We will also inspect whether a generator changes the base rate of the target in a way that makes downstream performance look fair only because the prediction problem became easier or less informative.

Deliverables and expected insight

To verify the evaluation chain, we will examine a small synthetic sample before computing headline metrics. We will check column types, allowed category values, missing entries, duplicate frequency, and whether the sensitive attribute and label appear in the expected format. We will then confirm that the downstream test rows never enter generator fitting and that the same test observations are scored for every method. Finally, the benchmark table will include sample counts and hardware details alongside wall-clock times, since isolated speed numbers are difficult to interpret without workload context. These checks make the fairness comparison reproducible and help locate failures before they become apparently meaningful results.

The project will provide scripts for data preparation, generator fitting and sampling, downstream evaluation, and plots, together with a record of versions and random seeds. The final report will relate the observed results to the paper's claims and separate numerical replication from any reduced-scope experiment. We expect to identify settings in which the tree-based approach is computationally attractive, and to learn whether fairness gains survive evaluation on untouched real data without an unacceptable loss of predictive utility. Mixed results are also scientifically useful: they can show that a particular dataset, classifier, or fairness definition changes the apparent ranking. The proposed workload is feasible on CPU hardware while retaining enough of the paper's evaluation chain to make the reproduction substantive.

We will retain the random seeds and generated samples used for every reported setting.

Papers to read

1. Emmanouil Panagiotou et al. "TABFAIRGDT: A Fast Fair Tabular Data Generator using Autoregressive Decision Trees." ICDM 2025. [Paper](#) · [Author implementation](#).